Longest Most Frequent Prefix

AlgoExpert · Hard · Tries

The problem

- Given unique strings, return the prefix shared by the most strings; ties go to the longest such prefix.
- Trie storing per-node counts.

Intuition

- A trie with counts at each node gives every prefix's frequency directly.

Brute force vs optimal

- Count all prefixes in a hash map → O(total chars²) strings
- Trie with counts → O(total chars)
- Track best by (count, length)

Approach

- 1. Insert strings counting passes per node
- 2. Track the max count and longest depth
- 3. Return that prefix

Worked example

```
algoexpert

algorithm

frontendexpert

mlexpert
```

Insert words into a trie with prefix counts

Worked example

prefix	count
a	2
al	2
alg	2
algo	2
algoe	1
f	1
m	1

Counts per prefix

Worked example

```
max count 2; longest such prefix: 'algo'
```

Answer: algo

Java solution

```java
public class LongestMostFrequentPrefix {
    private static class Node { final Map<Character, Node> next = new HashMap<>(); int count; }

    public static String longestMostFrequentPrefix(String[] strings) {
        Node root = new Node();
        int bestCount = 0;
        String best = "";
        for (String s : strings) {
            Node n = root;
            for (int i = 0; i < s.length(); i++) {
                n = n.next.computeIfAbsent(s.charAt(i), k -> new Node());
                n.count++;
            }
        }
        for (String s : strings) {
            Node n = root;
            for (int i = 0; i < s.length(); i++) {
                n = n.next.get(s.charAt(i));
                if (n.count > bestCount || (n.count == bestCount && i + 1 > best.length())) {
                    bestCount = n.count;
                    best = s.substring(0, i + 1);
                }
            }
        }
        return best;
    }
```

Complexity

- Time: O(n*m)
- Space: O(n*m)

Edge cases & pitfalls

- Tie → longest prefix
- Empty input → ''

Interview tips & follow-ups

- Explain trie node counts

Recap

- A trie with counts at each node gives every prefix's frequency directly.
- Complexity: O(n*m)